

Prompt Butter Jam vs. code2prompt

An evidence-based evaluation

Tested: 2026-08-02

Method: live browser instrumentation + local CLI benchmarks

Scope: vault (3,682 files) · 3 public repos · 1 synthetic edge-case repo

All PBJ testing used **synthetic, invented code only**.

No vault or private content was entered into any web form.

TL;DR — they are not competitors

	Prompt Butter Jam	code2prompt
Category	Prompt composer	Context packer
Input	A form you fill in	A directory tree
Output	~300-token instruction block	150K–10M-token code dump
Runs	Browser (client-side)	Local Rust binary
Maturity	Pre-release (core feature is a demo)	v4.3.0, stable
Verdict	Not usable yet	Production-ready, with caveats

They compose. PBJ writes the *instructions*; code2prompt supplies the *code*.
The real workflow is both — not either.

Headline finding ●

PBJ's "Sharpen" splices a canned example into your real prompt

I entered: a payment double-charge race condition (`charge_order` , idempotency key).

It returned a diff-merged hybrid containing text I never wrote:

```
GOAL:  
Fix aDiagnose why racethe conditionp95 double-charginglatency  
customers. CANARY...of OrdersController#index regressed after  
customer filtering was added...
```

`OrdersController#index` , p95 latency, slow-query logs, `EXPLAIN ANALYZE`
— **none of these were in my input.** They come from a hardcoded demo fixture.

Impact: the output is incoherent and unusable. Copy-pasting it sends an assistant after a bug that does not exist.

Why that matters more than it looks

The UI *does* disclose it — "Demo markup — the AI pass isn't live yet."

But the disclosure is **weaker than the affordance**:

- Button is the primary CTA, styled as the main action
- Output renders in the real preview pane, in the real format
- Change annotations (CLARIFIED , ASSUMED , ADDED) look like genuine analysis
- A hurried user copies a prompt about someone else's latency bug

Two clicks to reach it. Zero friction to copy it.

Verified: the privacy claim holds

Instrumented `fetch` , `XMLHttpRequest` , and `sendBeacon` ; injected a canary string (`CANARYSENTINEL7Z9QX`) and pasted fake AWS keys.

Check	Result
Prompt text transmitted	 No — zero leaks
Canary string in any request	 No
Cookies set	 None (<code>document.cookie</code> empty)
localStorage	Only <code>ps_tour_seen_v1</code>

Only one endpoint is called: `POST /api/events` .

Credit where due: the "runs entirely in your browser" claim is **true**, and I verified it rather than trusting it.

But the telemetry is more detailed than implied ⚠️

The site says *"no prompt content is sent anywhere."* Literally true.

What **is** sent on every sharpen:

```
{ "task_id": "8ffc7e99-8266-4587-8fa8-c5b38cfc3b30",  
  "event_name": "sharpen_requested",  
  "task_type": "debugging", "assistant": "copilot",  
  "model": "sonnet-4-5", "thinking_effort": "medium",  
  "prompt_chars": 1107, "context_chars": 334,  
  "estimated_saving_usd": 0.023622 }
```

Not content — but a **persistent** `task_id`, what you work on, and how big it is. Reasonable for analytics; worth stating plainly.

Bug: secrets pass through verbatim

Every generated prompt asserts:

```
EXCLUDE: Secrets, customer data, generated files, vendored dependencies...
```

I pasted into the context box:

```
AWS_SECRET_ACCESS_KEY=wJa1rXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY  
DB_PASSWORD=hunter2  
ghp_16CharsRealLookingToken0000000000
```

All three appear verbatim in the output prompt.

The `EXCLUDE:` line is an *instruction to the assistant*, not enforcement by the tool. It reads as a safety guarantee and isn't one — the most likely way a real user leaks a credential with this tool.

Bug: demo mode freezes the preview ●

Repro:

1. Fill the form, click *Sharpen · demo*
2. Edit the context textarea

Expected: preview updates. **Actual:** preview stays frozen on demo content while the textarea holds new text.

I nearly logged a false "secrets are redacted!" finding from this — the preview was showing stale demo output, not redacted input.

Also observed:

- *Sharpen* needs **two clicks** (confirm step) — first click looks like a no-op
- "**0 prompts sharpened**" counter never increments, even after sharpening
- Stale `Stack: Ruby on Rails, PostgreSQL` propagated into a prompt whose pasted code was **Python** — no validation against context

code2prompt: performance is excellent

Rust binary, v4.3.0. Best of 3 runs:

Target	Files	Tokens	Wall
synthetic edge repo	26	154	0.03s
obsidian vault	3,682	356K	0.08s
pallets/click	194	387K	0.10s
psf/requests	157	722K	0.24s
django/django	7,104	10M	1.19s

10M tokens from 7,104 files in **1.2 seconds**, ~248 MB RSS.
Speed is a non-issue. **Scale is the issue.**

code2prompt's real problem: the 10M-token cliff ●

Django produces **10M tokens** — roughly **50× any current context window**.

No warning is emitted. The tool reports the count and exits 0.

Narrowing it is manual work:

```
code2prompt repos/django -i "**/*.py" \  
  -e "**/tests/**" -e "**/*.lock"  
# → still 1M tokens
```

Even aggressive filtering leaves you **5× over budget**.

`--token-map` is the saving grace — it showed `uv.lock` alone was **31%** of click's tokens:

118K	— uv.lock		31%
110K	— tests		28%

code2prompt security: symlinks are followed by default ●

Synthetic repo containing `symlink_escape.txt -> /etc/passwd` :

```
code2prompt edge/repo -O out.md # no -L flag
```

Result — `/etc/passwd` contents inlined into the prompt:

```
`symlink_escape.txt`:
```txt
root:x:0:0:...
```

`--follow-symlinks` ( `-L` ) exists as an opt-in flag, but the contents are read **without it**. On an untrusted or vendored repo, a symlink pointing at `~/.ssh/id_rsa` or `~/.aws/credentials` is exfiltrated into whatever you paste into an LLM.

**Mitigation:** review the file tree before sending; avoid on untrusted repos.

## code2prompt: what it gets right

Same synthetic repo, default settings:

Hazard	Default behavior
<code>.env</code> with AWS keys	 <b>Excluded</b> (dotfile)
<code>node_modules/</code>	 <b>Excluded</b> (respects <code>.gitignore</code> )
3 MB binary <code>big.bin</code>	 <b>Body skipped</b> (name only; output 695 B)
Unicode/emoji filenames	 <b>Handled</b> ( <code>unicode_文件_🚀.py</code> )
Deep nesting (8 levels)	 Handled
Bad path / non-git <code>--diff</code>	 Clear error, <b>exit 1</b>

Note the sharp edge: `--hidden` **does** pull the `.env` AWS key into the output. The safe default is one flag away from an unsafe one.

## Install friction: a real finding ⚠️

Getting the *authentic* code2prompt was non-trivial:

Route	Outcome
<code>pip install code2prompt</code>	❌ PEP 668 blocks system install
<code>pip</code> in venv (Python 3.14)	❌ <code>tiktoken</code> wheel build fails — needs Rust
<code>npm install code2prompt</code>	⚠️ <b>Different, unrelated package</b> (v1.1.4)
<code>cargo install code2prompt</code>	✅ v4.3.0 in ~66s

**The npm name collision is a supply-chain trap** — it installs something else entirely under the expected name, with no error.

**Correct install:** `cargo install code2prompt` (Rust toolchain required).

PBJ, by contrast, installs in zero seconds — it's a web page.

## Where each one actually wins

**Prompt Butter Jam is right about something real.** Its structure —

GOAL / CONTEXT / SCOPE / EXCLUDE / CONSTRAINTS / OUTPUT / STOP WHEN —

is a genuinely good prompt skeleton. Explicit stop conditions and output shape *do* change assistant behavior.

The **checklist has value even with the product broken.**

**code2prompt wins** whenever the answer depends on code you'd otherwise paste by hand: cross-file refactors, "where is this used", onboarding.

**Neither** solves the actual hard problem — *selecting* the right 50K tokens from a 10M-token repo. code2prompt gives you a scalpel ( `-i` / `-e` / `--token-map` ) and leaves the judgment to you.

## Recommendation

**code2prompt — adopt**, with two standing rules:

1. Never run it on an untrusted repo without auditing symlinks
2. Always `--token-map` first; budget before you generate

**Prompt Butter Jam — do not use for real work yet.**

- Core "sharpen" feature is a hardcoded demo that corrupts your prompt
- `EXCLUDE: Secrets` is decorative, not enforced
- Revisit if/when the AI pass and codebase-aware sharpening ship

**Steal the skeleton anyway** — keep `GOAL / CONTEXT / CONSTRAINTS / OUTPUT / STOP WHEN` as a manual checklist. That's the transferable idea.

## Issues found — summary

#	Tool	Severity	Issue
1	PBJ	● High	Demo splices unrelated canned content into real prompts
2	PBJ	● High	<code>EXCLUDE: Secrets</code> claimed but not enforced — secrets pass through
3	c2p	● High	Symlinks read by default → <code>/etc/passwd</code> inlined
4	c2p	● Med	10M-token output, no context-limit warning
5	PBJ	● Med	Demo mode freezes preview; desyncs from input
6	npm	● Med	<code>npm i code2prompt</code> installs unrelated package



## Issues found — low severity

#	Tool	Severity	Issue
7	PBJ	● Low	"0 prompts sharpened" counter never increments
8	PBJ	● Low	Sharpen requires two clicks; first appears inert
9	PBJ	● Low	Stale <code>Stack</code> value contradicts pasted code's language
10	PBJ	● Low	Telemetry detail exceeds what the privacy note implies

**3 high · 3 medium · 4 low** — across both tools.

The three high-severity issues are each a *correctness or safety* problem, not a polish problem: a corrupted prompt, an unenforced safety claim, and an unintended file read.

## Method & reproducibility

**PBJ (browser):** patched `fetch` / `XHR` / `sendBeacon` to capture every outbound payload; canary-string leak detection; DOM assertions via `javascript_tool`. Only synthetic invented code was entered.

**code2prompt:** `cargo install code2prompt` → v4.3.0.

Timings = best of 3, `/usr/bin/time`. Corpus: this vault, `pallets/click`, `psf/requests`, `django/django`, plus a synthetic repo seeded with fake secrets, a 3 MB binary, unicode filenames, 8-level nesting, and a symlink to `/etc/passwd`.

## Caveats & corrections

**PBJ is a moving target** — an independent pre-release side project.

Findings are a snapshot of **2026-08-02**; the demo-splice bug (#1) should be re-checked before citing it anywhere.

**One correction made during testing:** an initial "secrets are redacted" reading was **wrong** — caused by the frozen-preview bug (#5), which showed stale demo output instead of the live input. Retracted after re-testing outside demo mode; the corrected finding is #2.

**One near-miss:** a suspected "exit 0 on error" bug in code2prompt was an artifact of `$?` capturing a piped `tail`, not the binary.

Exit codes are correct. Not counted as a finding.